DA - 2

Travel Itinerary and Booking System

25BCE1661 25BCE1693 25BCE1759

This report continues the DA1 travel-system design in Oracle. It documents the 14 implemented relations, populated data, 20 system-specific queries, PL/SQL objects, and insert, update and delete operations performed through the Java Swing interface and JDBC.

Problem statement:-

A user creates an itinerary containing destinations in a planned order. The user may travel with accompanying travellers and make hotel or flight bookings for a trip. Each booking has an associated payment record. The system stores service information and helps retrieve trip, cost, booking and payment details.

Table creation SQL

```
CREATE TABLE app_user (
user_id NUMBER(5) PRIMARY KEY,
name VARCHAR2(60) NOT NULL,
date_of_birth DATE NOT NULL,
gender VARCHAR2(15),
address VARCHAR2(150)
);

CREATE TABLE user_email (
user_id NUMBER(5) REFERENCES app_user(user_id),
email_id VARCHAR2(100),
PRIMARY KEY (user_id,email_id)
);

CREATE TABLE user_phone (
user_id NUMBER(5) REFERENCES app_user(user_id),
phone VARCHAR2(20),
PRIMARY KEY (user_id,phone)
);

CREATE TABLE traveller (
user_id NUMBER(5) REFERENCES app_user(user_id),
traveller_name VARCHAR2(60),
date_of_birth DATE NOT NULL,
gender VARCHAR2(15),
relationship_to_user VARCHAR2(30) NOT NULL,
PRIMARY KEY (user_id,traveller_name)
);
```

```
CREATE TABLE destination (
destination_id NUMBER(5) PRIMARY KEY,
city VARCHAR2(40) NOT NULL,
state VARCHAR2(40),
country VARCHAR2(40) NOT NULL,
description VARCHAR2(150),
base_cost NUMBER(10,2) NOT NULL CHECK (base_cost >= 0)
);

CREATE TABLE itinerary (
itinerary_id NUMBER(5) PRIMARY KEY,
user_id NUMBER(5) NOT NULL REFERENCES app_user(user_id),
trip_name VARCHAR2(60) NOT NULL,
start_date DATE NOT NULL,
end_date DATE NOT NULL,
total_estimated_cost NUMBER(10,2) NOT NULL CHECK (total_estimated_cost >= 0),
CHECK (end_date >= start_date)
);

CREATE TABLE itinerary_destination (
itinerary_id NUMBER(5) REFERENCES itinerary(itinerary_id),
destination_id NUMBER(5) REFERENCES destination(destination_id),
visit_order NUMBER(3) NOT NULL CHECK (visit_order > 0),
planned_days NUMBER(3) NOT NULL CHECK (planned_days > 0),
PRIMARY KEY (itinerary_id,destination_id),
UNIQUE (itinerary_id,visit_order)
);

CREATE TABLE booking (
booking_id NUMBER(5) PRIMARY KEY,
itinerary_id NUMBER(5) NOT NULL REFERENCES itinerary(itinerary_id),
booking_date DATE DEFAULT SYSDATE NOT NULL,
total_amount NUMBER(10,2) NOT NULL CHECK (total_amount >= 0),
booking_status VARCHAR2(15) DEFAULT 'PENDING' NOT NULL,
booking_type VARCHAR2(10) NOT NULL,
CHECK (booking_status IN ('PENDING','CONFIRMED','CANCELLED','COMPLETED')),
CHECK (booking_type IN ('HOTEL','FLIGHT'))
);

CREATE TABLE payment (
payment_id NUMBER(5) PRIMARY KEY,
booking_id NUMBER(5) NOT NULL UNIQUE REFERENCES booking(booking_id),
amount NUMBER(10,2) NOT NULL CHECK (amount >= 0),
payment_method VARCHAR2(15),
payment_date DATE,
payment_status VARCHAR2(15) DEFAULT 'INITIATED' NOT NULL,
transaction_id VARCHAR2(40) UNIQUE,
CHECK (payment_method IN ('UPI','CARD','CASH','NET_BANKING','WALLET')),
CHECK (payment_status IN ('INITIATED','SUCCESS','FAILED','REFUNDED')),
CHECK (payment_status NOT IN ('SUCCESS','REFUNDED') OR
(payment_method IS NOT NULL AND payment_date IS NOT NULL AND transaction_id IS NOT
NULL))
);
```

```
CREATE TABLE hotel (
hotel_id NUMBER(5) PRIMARY KEY,
destination_id NUMBER(5) NOT NULL REFERENCES destination(destination_id),
hotel_name VARCHAR2(60) NOT NULL,
address VARCHAR2(150),
contact_number VARCHAR2(20),
rating NUMBER(2,1) NOT NULL CHECK (rating BETWEEN 0 AND 5),
base_price NUMBER(10,2) NOT NULL CHECK (base_price >= 0)
);

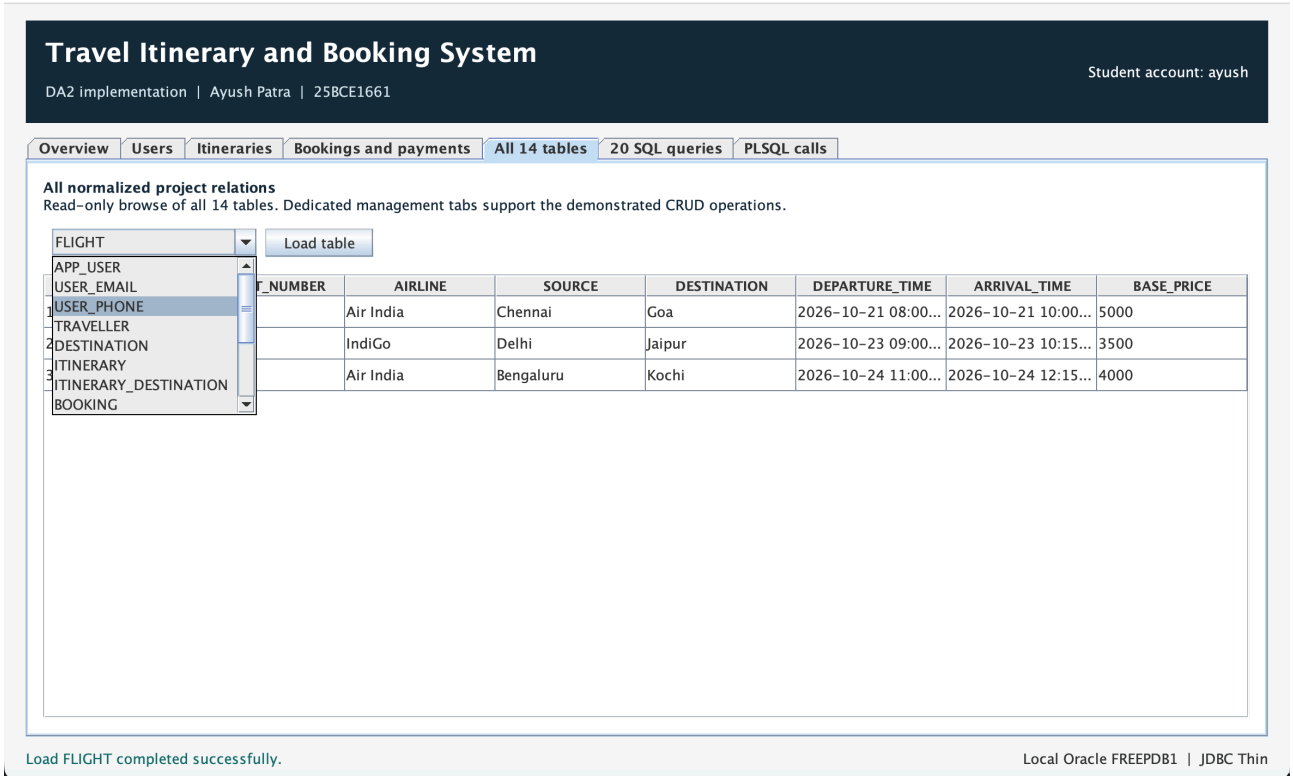
CREATE TABLE hotel_booking (
booking_id NUMBER(5) PRIMARY KEY REFERENCES booking(booking_id),
hotel_id NUMBER(5) NOT NULL REFERENCES hotel(hotel_id),
check_in_date DATE NOT NULL,
check_out_date DATE NOT NULL,
number_of_guests NUMBER(3) NOT NULL CHECK (number_of_guests > 0),
CHECK (check_out_date > check_in_date)
);

CREATE TABLE hotel_booked_room (
booking_id NUMBER(5) REFERENCES hotel_booking(booking_id),
room_number VARCHAR2(10),
PRIMARY KEY (booking_id,room_number)
);

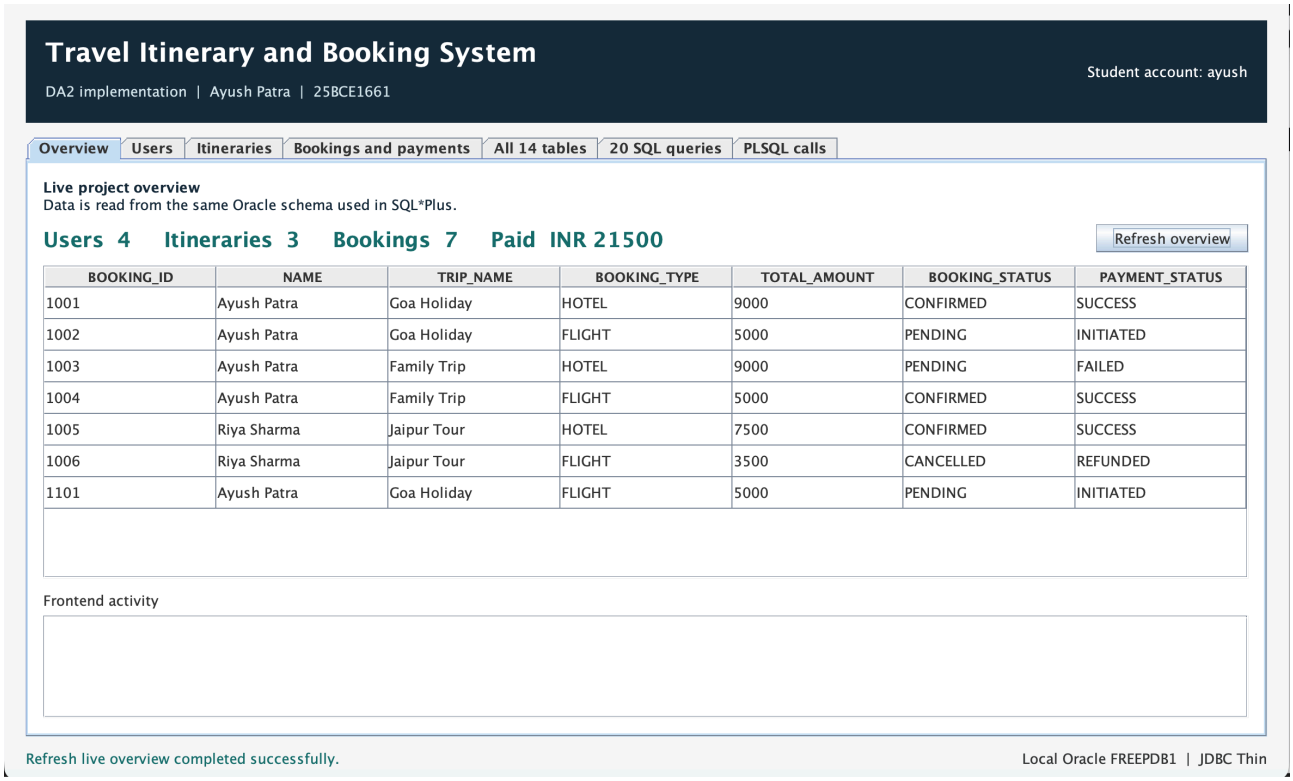
CREATE TABLE flight (
flight_id NUMBER(5) PRIMARY KEY,
flight_number VARCHAR2(15) NOT NULL,
airline VARCHAR2(40) NOT NULL,
source VARCHAR2(40) NOT NULL,
destination VARCHAR2(40) NOT NULL,
departure_time TIMESTAMP NOT NULL,
arrival_time TIMESTAMP NOT NULL,
base_price NUMBER(10,2) NOT NULL CHECK (base_price >= 0),
CHECK (arrival_time > departure_time),
CHECK (source <> destination)
);

CREATE TABLE flight_booking (
booking_id NUMBER(5) PRIMARY KEY REFERENCES booking(booking_id),
flight_id NUMBER(5) NOT NULL REFERENCES flight(flight_id),
seat_number VARCHAR2(10) NOT NULL,
seat_class VARCHAR2(20) NOT NULL,
UNIQUE (flight_id,seat_number),
CHECK (seat_class IN ('ECONOMY','BUSINESS','FIRST','PREMIUM_ECONOMY'))
);
```

Relation	Primary key	Foreign key mapping
APP_USER	user_id	None
USER_EMAIL	user_id and email_id	user_id to APP_USER
USER_PHONE	user_id and phone	user_id to APP_USER
TRAVELLER	user_id and traveller_name	user_id to APP_USER
DESTINATION	destination_id	None
ITINERARY	itinerary_id	user_id to APP_USER
ITINERARY_ DESTINATION	itinerary_id and destination_id	IDs to ITINERARY and DESTINATION
BOOKING	booking_id	itinerary_id to ITINERARY
PAYMENT	payment_id	booking_id to BOOKING and unique
HOTEL	hotel_id	destination_id to DESTINATION
HOTEL_BOOKING	booking_id	booking_id to BOOKING and hotel_id to HOTEL
HOTEL_BOOKED_ ROOM	booking_id and room_number	booking_id to HOTEL_BOOKING
FLIGHT	flight_id	None
FLIGHT_BOOKING	booking_id	booking_id to BOOKING and flight_id to FLIGHT



Frontend Overview



Sample Data Insertion:-

iNSERT INTO app_user VALUES (1,'Ayush Patra',DATE '2005-04-15','Male','Chennai');
INSERT INTO itinerary_destination VALUES (1,1,1,3);
INSERT INTO flight_booking VALUES (1002,1,'11A','ECONOMY');
INSERT INTO payment VALUES (5001,1001,9000,'UPI',DATE
'2026-10-02','SUCCESS','TXN1001');
COMMIT;

System Queries:-

Q1 ORDER BY users alphabetically

```
SELECT user_id,name,address FROM app_user ORDER BY name;
```

User Id	Name	Address
3	Arjun Rao	Bengaluru
1	Ayush Patra	Chennai
4	Meera Nair	Kochi
2	Riya Sharma	Delhi

Q2 Comparison AND BETWEEN suitable hotels

```
SELECT hotel_name,rating,base_price FROM hotel
WHERE rating>=4 AND base_price BETWEEN 2000 AND 3500 ORDER BY base_price;
```

Hotel Name	Rating	Base Price
Pink City Hotel	4.8	2500
Goa Bay Hotel	4.5	3000

Q3 LIKE and string functions

```
SELECT name,UPPER(name) AS upper_name,LENGTH(name) AS name_length,
       SUBSTR(name,1,5) AS short_name FROM app_user WHERE name LIKE 'A%';
```

Name	Upper Name	Name Length	Short Name
Ayush Patra	AYUSH PATRA	11	Ayush
Arjun Rao	ARJUN RAO	9	Arjun

Actual output: 2 rows.

Q4 Arithmetic and ROUND destination cost with 5 percent buffer

```
SELECT city,base_cost,ROUND(base_cost*1.05,2) AS buffered_cost FROM destination;
```

City	Base Cost	Buffered Cost
Panaji	4000	4200
Jaipur	3000	3150
Kochi	3500	3675
Manali	5000	5250

Actual output: 4 rows.

Q5 Oracle date functions and date subtraction

```
SELECT trip_name,TO_CHAR(start_date,'DD-MON-YYYY') AS start_on,
       TRUNC(end_date)-TRUNC(start_date)+1 AS trip_days,
       EXTRACT(YEAR FROM start_date) AS trip_year FROM itinerary;
```

Trip Name	Start On	Trip Days	Trip Year
Goa Holiday	20-OCT-2026	6	2026
Family Trip	20-OCT-2026	6	2026
Jaipur Tour	22-OCT-2026	6	2026

Actual output: 3 rows.

Q6 GROUP BY and aggregate functions

```
SELECT booking_type,COUNT(*) AS bookings,SUM(total_amount) AS total_value,
       ROUND(AVG(total_amount),2) AS average_value,
       MIN(total_amount) AS minimum_value,MAX(total_amount) AS maximum_value
FROM booking GROUP BY booking_type;
```

Booking Type	Bookings	Total Value	Average Value	Minimum Value	Maximum Value
HOTEL	3	25500	8500	7500	9000
FLIGHT	4	18500	4625	3500	5000

Actual output: 2 rows.

Q7 HAVING destinations in multiple itineraries

```
SELECT d.city,COUNT(*) AS itinerary_count FROM destination d
JOIN itinerary_destination x ON x.destination_id=d.destination_id
GROUP BY d.destination_id,d.city HAVING COUNT(*)>1;
```

City	Itinerary Count
Panaji	2
Jaipur	2
Kochi	2

Actual output: 3 rows.

Q8 INNER JOIN itinerary owners

```
SELECT u.name,i.trip_name FROM app_user u
INNER JOIN itinerary i ON i.user_id=u.user_id ORDER BY u.user_id,i.itinerary_id;
```

Name	Trip Name
Ayush Patra	Goa Holiday
Ayush Patra	Family Trip
Riya Sharma	Jaipur Tour

Actual output: 3 rows.

Q9 LEFT JOIN also include users without itineraries

```
SELECT u.name,i.trip_name FROM app_user u
LEFT JOIN itinerary i ON i.user_id=u.user_id ORDER BY u.user_id;
```

Name	Trip Name
Ayush Patra	Goa Holiday
Ayush Patra	Family Trip
Riya Sharma	Jaipur Tour
Arjun Rao	NULL

Actual output: 5 rows. First 4 shown; the evidence JSON contains the full result.

Q10 RIGHT JOIN also include destinations without hotels

```
SELECT d.city,h.hotel_name FROM hotel h
RIGHT JOIN destination d ON d.destination_id=h.destination_id ORDER BY d.destination_id;
```

City	Hotel Name
Panaji	Goa Bay Hotel
Panaji	Beach Resort
Jaipur	Pink City Hotel
Kochi	NULL

Actual output: 5 rows. First 4 shown; the evidence JSON contains the full result.

Q11 FULL JOIN contact combinations including missing email or phone

```
SELECT NVL(e.user_id,p.user_id) AS user_id,e.email_id,p.phone
FROM user_email e FULL OUTER JOIN user_phone p ON e.user_id=p.user_id
ORDER BY user_id;
```

User Id	Email Id	Phone
1	ayush.work@example.com	9123456780
1	ayush@example.com	9123456780
1	ayush.work@example.com	9876543210
1	ayush@example.com	9876543210

Actual output: 7 rows. First 4 shown; the evidence JSON contains the full result.

Q12 CROSS JOIN possible user and destination combinations

```
SELECT u.name,d.city FROM app_user u CROSS JOIN destination d ORDER BY
  u.user_id,d.destination_id;
```

Name	City
Ayush Patra	Panaji
Ayush Patra	Jaipur
Ayush Patra	Kochi
Ayush Patra	Manali

Actual output: 16 rows. First 4 shown; the evidence JSON contains the full result.

Q13 SELF JOIN pairs of hotels in the same destination

```
SELECT h1.hotel_name AS hotel_one,h2.hotel_name AS hotel_two
FROM hotel h1 JOIN hotel h2 ON h1.destination_id=h2.destination_id AND
  h1.hotel_id<h2.hotel_id;
```

Hotel One	Hotel Two
Goa Bay Hotel	Beach Resort

Actual output: 1 rows.

Q14 NATURAL JOIN users with phone numbers

```
SELECT user_id,name,phone FROM app_user NATURAL JOIN user_phone ORDER BY user_id,phone;
```

User Id	Name	Phone
1	Ayush Patra	9123456780
1	Ayush Patra	9876543210
2	Riya Sharma	9876543211
4	Meera Nair	9876543213

Actual output: 4 rows.

Q15 JOIN USING user email addresses

```
SELECT user_id,name,email_id FROM app_user JOIN user_email USING(user_id) ORDER BY
  user_id,email_id;
```

User Id	Name	Email Id
1	Ayush Patra	ayush.work@example.com
1	Ayush Patra	ayush@example.com
2	Riya Sharma	riya@example.com
3	Arjun Rao	arjun@example.com

Actual output: 4 rows.

Q16 EQUI JOIN using WHERE bookings and trips

```
SELECT b.booking_id,i.trip_name,b.total_amount FROM booking b,itinerary i
WHERE b.itinerary_id=i.itinerary_id ORDER BY b.booking_id;
```

Booking Id	Trip Name	Total Amount
1001	Goa Holiday	9000
1002	Goa Holiday	5000
1003	Family Trip	9000
1004	Family Trip	5000

Actual output: 7 rows. First 4 shown; the evidence JSON contains the full result.

Q17 NON EQUI JOIN hotel rates within a trip daily budget

```
SELECT i.trip_name,h.hotel_name,h.base_price FROM itinerary i
JOIN hotel h ON
  h.base_price<=i.total_estimated_cost/(TRUNC(i.end_date)-TRUNC(i.start_date)+1)
ORDER BY i.itinerary_id,h.hotel_id;
```

Trip Name	Hotel Name	Base Price
Goa Holiday	Goa Bay Hotel	3000
Goa Holiday	Pink City Hotel	2500
Family Trip	Goa Bay Hotel	3000
Family Trip	Beach Resort	4500

Actual output: 7 rows. First 4 shown; the evidence JSON contains the full result.

Q18 Scalar subquery bookings above average cost

```
SELECT booking_id,total_amount FROM booking
WHERE total_amount>(SELECT AVG(total_amount) FROM booking) ORDER BY total_amount DESC;
```

Booking Id	Total Amount
1001	9000
1003	9000
1005	7500

Actual output: 3 rows.

Q19 IN subquery destinations with highly rated hotels

```
SELECT city FROM destination WHERE destination_id IN
  (SELECT destination_id FROM hotel WHERE rating>=4.5);
```

City
Panaji
Jaipur

Actual output: 2 rows.

Q20 Multiple joins failed payment booking owners

```
SELECT u.name,i.trip_name,b.booking_id,p.payment_status
FROM app_user u JOIN itinerary i ON i.user_id=u.user_id
JOIN booking b ON b.itinerary_id=i.itinerary_id
JOIN payment p ON p.booking_id=b.booking_id WHERE p.payment_status='FAILED';
```

Name	Trip Name	Booking Id	Payment Status
Ayush Patra	Family Trip	1003	FAILED

Actual output: 1 rows.

PLSQL Programming

Topic	Implemented demonstration
Variables and data types	NUMBER VARCHAR2 BOOLEAN constants %TYPE and %ROWTYPE.
Conditional statements	IF ELSE ELSIF and CASE using travel budgets and payment status.
Loops	Basic LOOP WHILE FOR REVERSE FOR and CONTINUE.
Procedures and parameters	IN user details; OUT booking count; IN OUT budget buffer.
Functions	Trip-day calculation and successful user spending.
Implicit cursor	SQL%ROWCOUNT SQL%FOUND and SQL%NOTFOUND after DML.
Explicit cursor	OPEN FETCH %NOTFOUND %ROWCOUNT and CLOSE for bookings.
Parameterized cursor	Cursor FOR loop and row records filtered by user.
Cursor update	FOR UPDATE and WHERE CURRENT OF with rollback.
Exceptions	NO_DATA_FOUND DUP_VAL_ON_INDEX user-defined exceptions OTHERS and RAISE_APPLICATION_ERROR.
Triggers	BEFORE AFTER row-level statement-level conditional predicates and old/new row values.
Views	Simple joined aggregate and WITH CHECK OPTION views.

Procedures:-

```
CREATE OR REPLACE PROCEDURE tc_booking_count(p_user IN NUMBER,p_count OUT NUMBER) IS
BEGIN
  SELECT COUNT(*) INTO p_count FROM booking b JOIN itinerary i ON
  i.itinerary_id=b.itinerary_id
  WHERE i.user_id=p_user;
END;

CREATE OR REPLACE PROCEDURE tc_add_buffer(p_budget IN OUT NUMBER,p_buffer IN NUMBER) IS
BEGIN
  IF p_budget IS NULL OR p_buffer IS NULL OR p_budget<0 OR p_buffer<0 THEN
    RAISE_APPLICATION_ERROR(-20011,'Budget and buffer must be non-negative.');
  END IF;
  p_budget:=p_budget+p_buffer;
END;

CREATE OR REPLACE FUNCTION tc_trip_days(p_itinerary IN NUMBER) RETURN NUMBER IS
  v_days NUMBER;
BEGIN
  SELECT TRUNC(end_date)-TRUNC(start_date)+1 INTO v_days
  FROM itinerary WHERE itinerary_id=p_itinerary;
  RETURN v_days;
EXCEPTION
  WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-20012,'Itinerary not found.');
END;
```

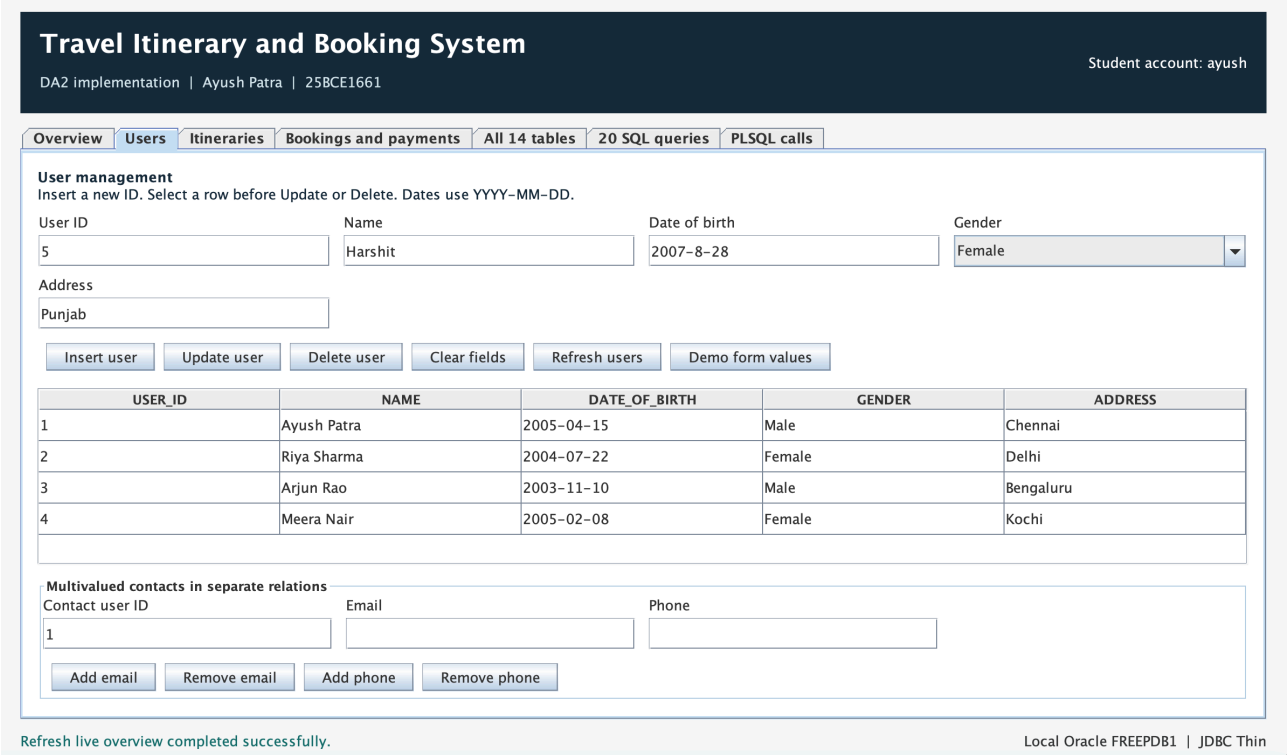
Triggers and database views

Trigger	Timing and action
TC_BOOKING_TYPE	BEFORE row update prevents a change of booking type.
TC_HOTEL_TYPE	BEFORE row insert/update checks the parent HOTEL type.
TC_FLIGHT_TYPE	BEFORE row insert/update checks the parent FLIGHT type.
TC_PAYMENT_CHECK	BEFORE row insert/update validates amount dates payment state and booking state.
TC_PAYMENT_AFTER	AFTER row insert/update synchronizes booking confirmation or cancellation.
TC_USER_MESSAGE	AFTER row DML uses INSERTING UPDATING DELETING to print messages.
TC_HOTEL_STATEMENT	AFTER statement hotel update prints a completion message.
TC_BOOKING_DELETE	BEFORE row delete rejects CONFIRMED or COMPLETED bookings.

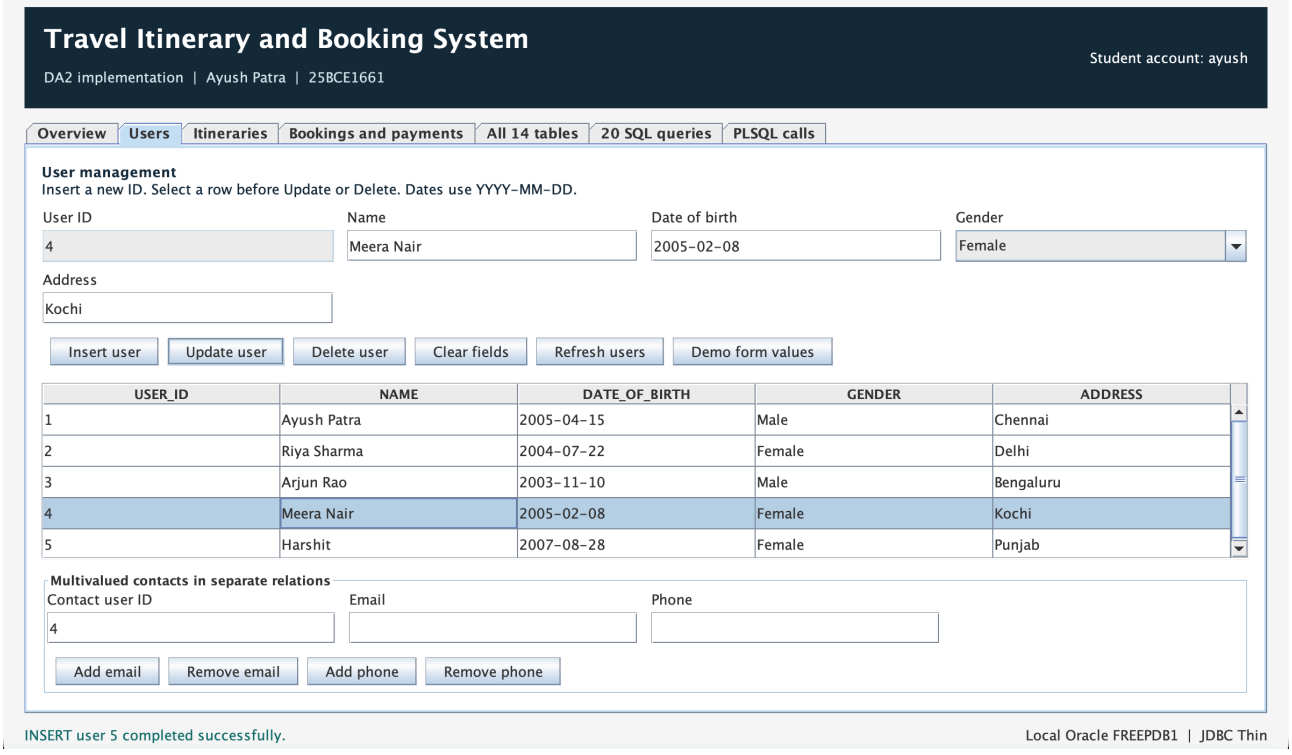
```
CREATE OR REPLACE TRIGGER tc_payment_after
AFTER INSERT OR UPDATE OF payment_status ON payment FOR EACH ROW
BEGIN
  IF :NEW.payment_status='SUCCESS' THEN
    UPDATE booking SET booking_status='CONFIRMED'
    WHERE booking_id=:NEW.booking_id AND booking_status='PENDING';
  ELSIF :NEW.payment_status='REFUNDED' THEN
    UPDATE booking SET booking_status='CANCELLED' WHERE booking_id=:NEW.booking_id;
  END IF;
END;
```

View	Purpose
TC_V_USERS	Simple projection of user details.
TC_V_BOOKINGS	Joins user itinerary booking and payment for the frontend.
TC_V_REVENUE	Aggregates successful payments per user.
TC_V_BUDGET_HOTELS	Hotels priced at most INR 3500 with a view CHECK OPTION.

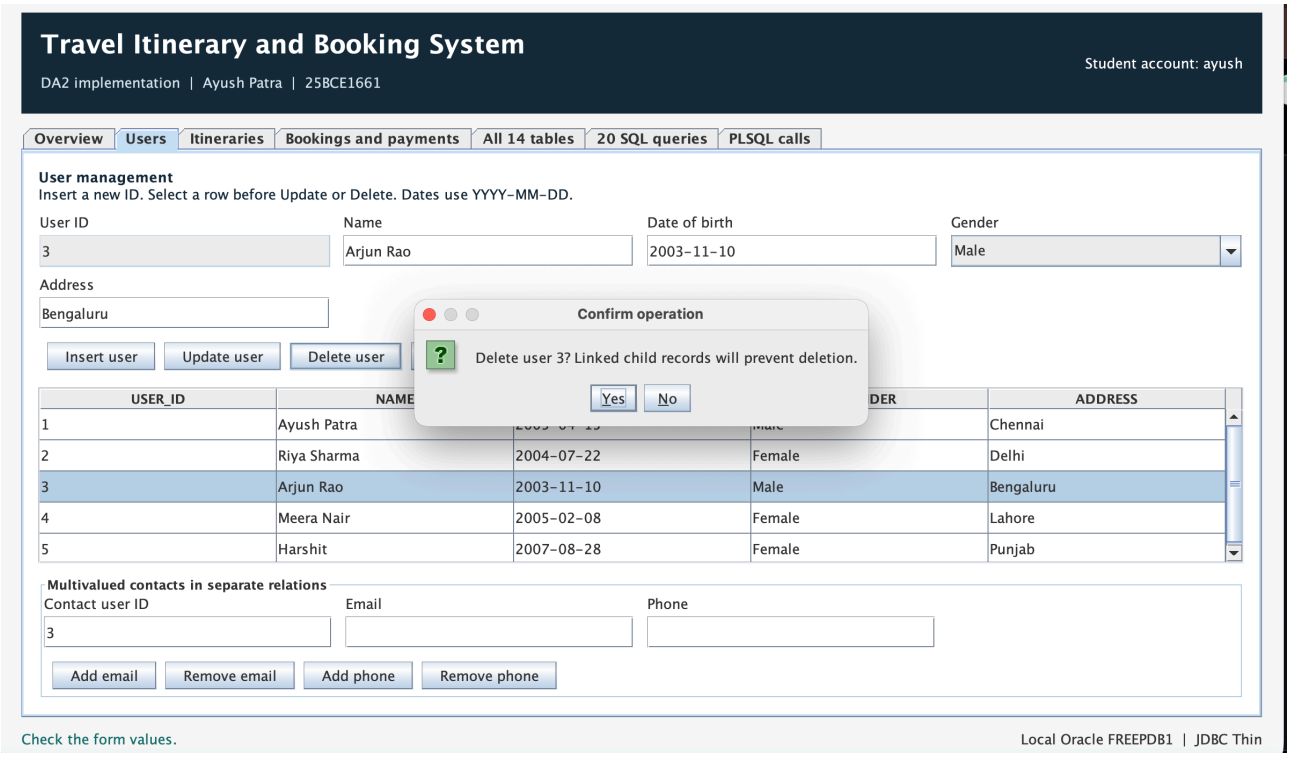
Insert through the interface



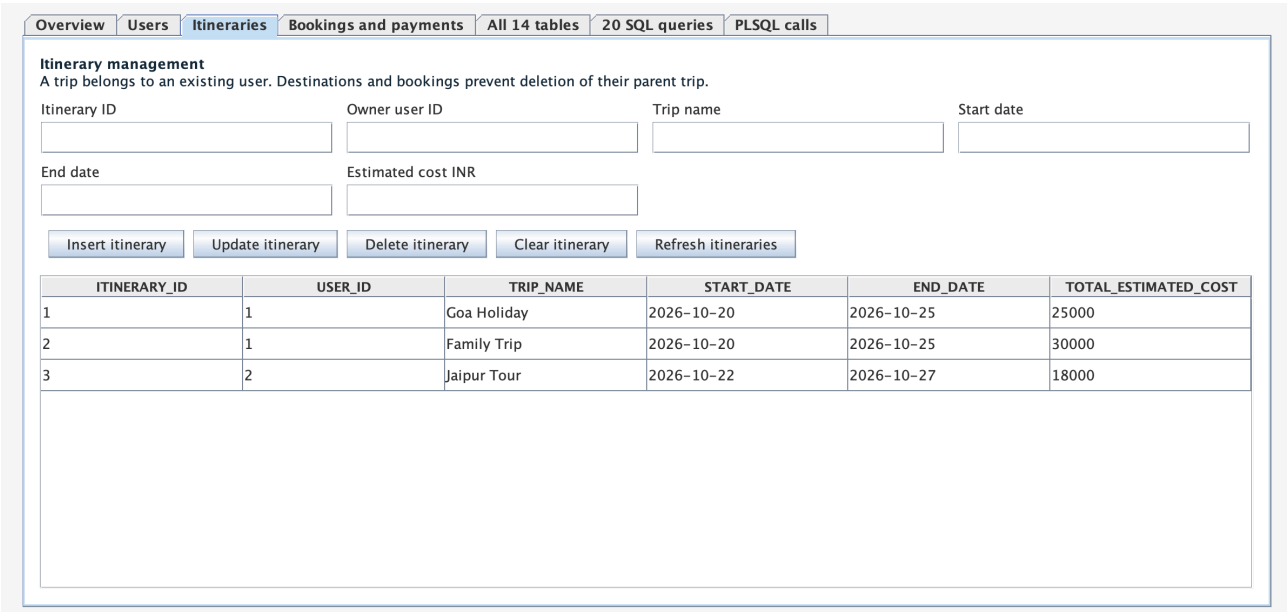
Update through the interface



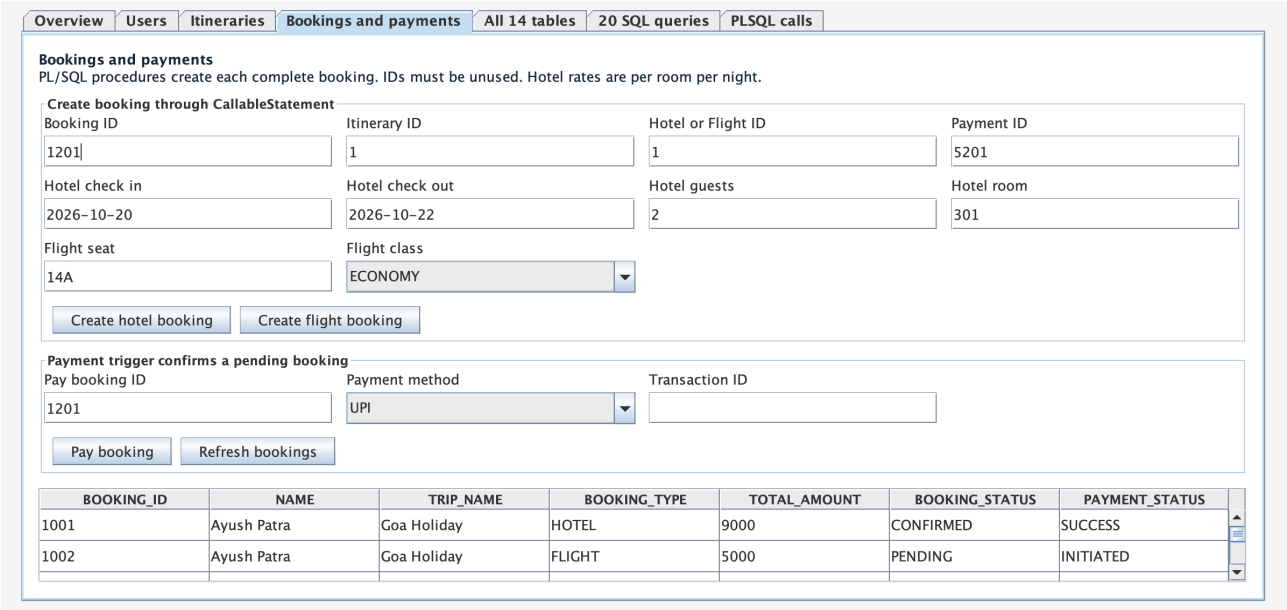
Delete through the interface



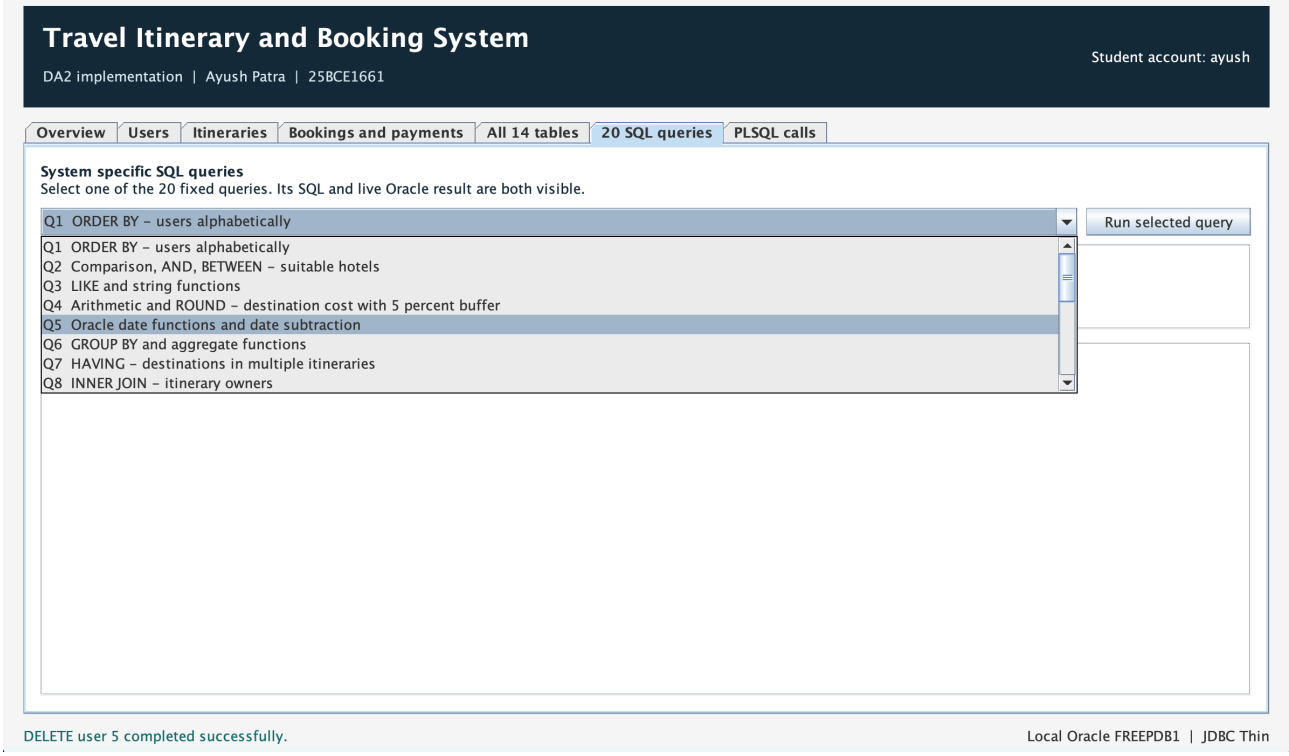
Itinerary Insert, Update and Delete



Booking and Payment Insert, Update and Delete



SQL query through the interface



PLSQL calls through the interface

